Вариант 13 • NoSQL

Интеграционное тестирование СУБД NoSQL в Docker

Проектирование и тестирование NoSQL-подсистемы «GPS-события транспорта» на базе Apache Cassandra

Высокопроизводительное хранение и анализ временных рядов (Time-Series) телеметрии автопарка с использованием Docker и Python

СТУДЕНТ

Абдыкапаров Даниял

ГРУППА

ИС24/22



ПРОБЛЕМАТИКА И ТРЕБОВАНИЯ

Контекст задачи: Телеметрия коммерческого автопарка

Высокий поток данных

Сотни ТС передают координаты каждые 1–5 секунд. Пиковая нагрузка требует высокой скорости записи.

Time-Series специфика

Запросы строятся строго вокруг отрезков времени и идентификаторов транспортных средств (TS + Date).

Отказоустойчивость

Необходима непрерывная доступность на запись без единой точки отказа (Single Point of Failure).

Главная цель: Построить схему Cassandra с суточным бакетированием, обеспечить батч-запись телеметрии и покрыть систему автоматизированными интеграционными тестами.

ПРОЕКТИРОВАНИЕ ОТ ЗАПРОСОВ

Query-Driven Modeling: Шаблоны доступа

ID	Описание шаблона (Access Pattern)	Целевой CQL-запрос
AP-1	Запись точки телеметрии	`INSERT INTO vehicle_gps_tracks...`
AP-2	Получение трека за конкретную дату	`WHERE vehicle_id = ? AND date = ?`
AP-3	Фильтрация трека по интервалу времени	`...AND timestamp ⩾ ? AND timestamp ⩽ ?`
AP-4	Получение последней локации (Last Position)	`...WHERE vehicle_id = ? LIMIT 1`
AP-5	Анализ превышения скорости (> limit)	`Фильтрация на стороне клиенту / ALLOW FILTERING`

Золотое правило Cassandra

В отличие от реляционных БД, таблица создается под конкретные запросы. Мы избегаем соединений (`JOIN`) и сканирования всей таблицы (`FULL SCAN`). Каждая запись точно адресуется партиционным ключом.

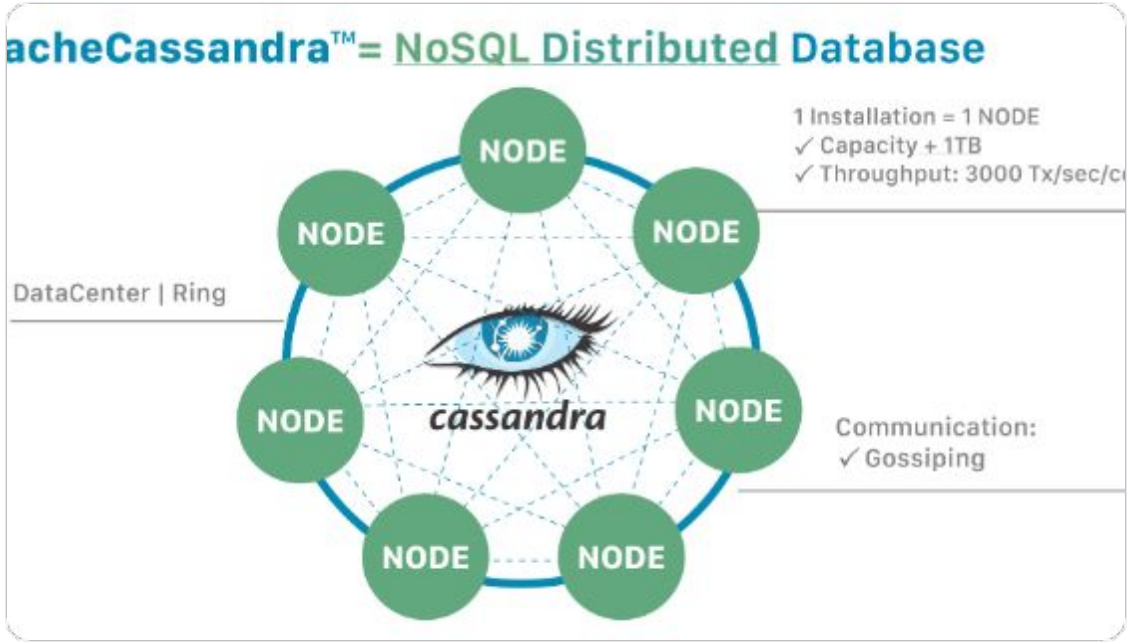
АРХИТЕКТУРНЫЙ ВЫБОР

Почему Apache Cassandra? (ADR)

1. **LSM-Tree Движок записи**
   Запись происходит последовательно в MemTable и CommitLog ($O(1)$), что дает колоссальное преимущество перед B-Tree в RDBMS.

2. **Децентрализация (Masterless)**
   Все узлы равноправны. При падении любой ноды запись продолжает функционировать без задержек на выборы лидера.

3. **Суточное бакетирование & TTL**
   Встроенная поддержка автоматического удаления устаревших логов без ресурсоемких операций . `DELETE`



СТРУКТУРА ДАННЫХ

CQL-схема: Таблица vehicle_gps_tracks

```
CREATE KEYSPACE IF NOT EXISTS fleet_monitoring
WITH replication = {
   'class': 'SimpleStrategy',
   'replication_factor': 1
 };

CREATE TABLE fleet_monitoring.vehicle_gps_tracks (
   vehicle_id text,
   date date,
   timestamp timestamp,
   latitude double,
   longitude double,
   speed double,
   altitude double,
   status text,
   PRIMARY KEY ((vehicle_id, date), timestamp)
 ) WITH CLUSTERING ORDER BY (timestamp DESC);
```

Partition Key: (vehicle_id, date)

Составной ключ гарантирует, что все точки одного авто за 1 день попадают на одну ноду. Размер партиции остается оптимальным (<100 МБ).

Clustering Key: timestamp DESC

Отвечает за физическую сортировку на диске. позволяет моментально запрашивать последние координаты с помощью .

DESC

LIMIT 1

РЕАЛИЗАЦИЯ НА PYTHON

Архитектура Python-модуля и драйвер

```
# Структура репозитория

project_root/
  ├── asyncore.py # Заглушка для Python 3.12+
  ├── data/
  │   └── generate_data.py # Симуляция телеметрии
  ├── src/
  │   └── queries.py # Драйвер GPSHarness
  └── tests/
      └── test_tracking.py # Интеграционные тесты
```

Класс GPSHarness (src/queries.py)

Инкапсулирует подключение через `Cluster(['127.0.0.1'])`, подготовку подготовленных выражений (Prepared Statements) и выполнение пакетных вставок.

Решенная проблема совместимости

В Python 3.12+ удален модуль `asyncore`. Для корректной работы создана кастомная заглушка в корне проекта.

ИНФРАСТРУКТУРА И ДАННЫЕ

Развертывание в Docker и Генерация данных

```
# 1. Запуск контейнера Cassandra
docker run --name cassandra-node \
   -p 9042:9042 -d cassandra:latest

# 2. Генерация тестовых треков
python -m data.generate_data
```

Результаты работы симулятора телеметрии

- **Сгенерировано:** 36 фиктивных GPS-точек
- **Транспорт:** BUS-101, TAXI-202, TRUCK-303
- **Атрибуты:** Широта, долгота, скорость (0-90 км/ч), статус
- **Оптимизация:** Использование `BatchStatement`

Порт 9042 проброшен локально для подключения Python драйвера к Docker-контейнеру.

ПРОВЕРКА ДАННЫХ

Консольный контроль через CLI (cqlsh)

```
cqlsh> SELECT vehicle_id, date, timestamp, speed, status FROM fleet_monitoring.vehicle_gps_tracks LIMIT 5;

 vehicle_id | date | timestamp | speed | status
------------+------------+---------------------------------+-------+---------
     BUS-101 | 2026-03-30 | 2026-03-30 14:22:10.000000+0000 | 62.4 | MOVING
     BUS-101 | 2026-03-30 | 2026-03-30 14:21:10.000000+0000 | 58.1 | MOVING
    TAXI-202 | 2026-03-30 | 2026-03-30 14:20:00.000000+0000 | 0.0 | STOPPED
   TRUCK-303 | 2026-03-30 | 2026-03-30 14:18:00.000000+0000 | 84.2 | MOVING

(4 rows affected)
```

Подтверждение кластеризации
Точки автоматически сгруппированы по дате и отсортированы по убыванию времени.

Целостность типом данных
Типы , double timestamp text и запись в без искажений

КАЧЕСТВО И ТЕСТЫ

Интеграционное тестирование (unittest)

```
# Запуск тестов в консоли
python -m unittest tests/test_tracking.py

test_get_last_position (tests.test_tracking.TestGPSTracking) ... ok
test_speeding_alerts (tests.test_tracking.TestGPSTracking) ... ok

----------------------------------------------------------------------
Ran 2 tests in 1.482s - OK
```

Test 1: Last Position

Проверяет корректность выборки последней точки для заданного с помощью . LIMIT 1

Test 2: Speeding Alerts

Валидирует бизнес-логику фильтрации событий, где скорость превышает заданный порог (например, > 80 км/ч).

ИТОГИ РАБОТЫ

Заключение и Перспективы развития

Цели достигнуты

Разработана отказоустойчивая NoSQL подсистема хранения телеметрии. Проведены интеграционные тесты.

Оптимизация производительности

Использование суточных бакетов минимизирует размер партиций и исключает перегрузку отдельных нод.

Развитие проекта

Интеграция с Apache Spark для гео-аналитики в реальном времени и настройка мультинодного кластера в Docker Compose.

Спасибо за внимание!

Даниял Абдыкапаров • ИС24/22